Machine Translated by Google

Deploying an AI server

in Astra Linux 1.7 OS

to preserve text formatting when working with a document

(copy/paste fragments) use Chromium and Yandex

browsers

ollamA-webul AI

The educational material was provided by:

https://github.com/UAR-2020/ www.uar@gmail.com

2026

Machine Translated by Google

Content:

I. Creation of the necessary software environment

On test server 1: 1. Installing the

Ollama server (Nvidia is not required)……..…….3 2. Creating a pool of necessary models for the

Ollama server……..…8 3. Creating a portable archive of models for the Ollama

server…….…... 9 4. Downloading and archiving the docker with the Ollama

server……..….. 10

5. Downloading and archiving Docker with a WebUI server……..10

6. Downloading NVIDIA Container Toolkit…….... 10

II. Deployment of the created software environment On test server 2: 1. installation of

docker.io and installation of the

driver and NVIDIA Container Toolkit……..... 11

2. Unpacking and loading (loading) the docker with the Ollama server…….. 12 3.

Unpacking the portable archive of models for the docker with the Ollama server…....12 4. Unpacking and

loading (loading) the docker with the WebUI server……..……13 5. Running (running) docker

with the Ollama server and WebUI server…… 13

III. Connecting to the created AI server via a browser On any test workstation:

1. Connecting to the AI

server……17 2. Operating

principle and setting up models…….. 17 3. Resolving

error 500 on the WebUI client……17

IV. Creating and connecting the AI assistant cli to the AI server

On any test workstation: 1. creating and adding a script to

~/.bashrc……..…….. 18 2. calling the client and the basics of working with

it…..…….. 19

V. Other 1.

Disabling reasoning for models via WebUI and CLI…… 21

2. Removing dockers and their data…….. 22

3. Limiting RAM consumption by dockers…….... 23

ollamA-webui AI

Machine Translated by Google

On test server 1:

Ollama installation instructions are taken from the URL resource

https://docs.ollama.com/linux

Installing Ollama utomatically download and configure the service)

To install Ollama, run the following command:

curl -fsSL https://ollama.com/install.sh | sh

Manual installation of Ollama

[*** all versions of Ollama are located at

https://github.com/ollama/ollama/releases ,

Please take this into account when creating the address when downloading the package (!!!)

]

If you are upgrading from a previous version, you should first remove the old ones.
libraries sudo rm -rf /usr/lib/ollama.

Download and unzip the package:

curl -fsSL https://ollama.com/download/ollama-linux-
amd64.tar.zst \

| sudo tar x -C /usr

Launch of ollama:

ollama serve

In another terminal, make sure Ollama is running:

ollama -v

Adding Ollama as a service (recommended)

Create a user and group for Ollama:

ollamA-webul AI

Machine Translated by Google

```
sudo useradd -r -s /bin/false -U -m -d /usr/share/ollama ollama
sudo usermod -a -G ollama $(whoami)
```

Create a service file in /etc/systemd/system/ollama.service:

```
[Unit]
Description=Ollama Service
After=network-online.target

[Service]
ExecStart=/usr/bin/ollama serve
User=ollama
Group=ollama
Restart=always
RestartSec=3
Environment="PATH=$PATH"

[Install]
WantedBy=multi-user.target
```

Then start the service:

```
sudo systemctl daemon-reload
sudo systemctl enable ollama
```

Installing CUDA drivers (optional).

Download and install CUDA.Make sure the drivers are installed by running the following command, which should output detailed information about your video card:

```
nvidia-smi
```

ollamA-webul AI

Machine Translated by Google

When installing from apt, select the required version, for example:

```
sudo apt install nvidia-driver-575
```

In our case there is no need for installation proprietary drivers instead of Nouveau from Astra Linux due to the fact that only the necessary models will be downloaded for Ollama to create the necessary software environment

Launch of Ollama

Launch Ollama and make sure it works:

```
sudo systemctl start ollama
sudo systemctl status ollama
```

Setting up

To customize your Ollama installation, you can edit the systemd service file or environment variables by running the command:

```
sudo systemctl edit ollama
```

Alternatively, create an override file manually in /etc/systemd/system/ollama.service.d/override.conf:

```
[Service]
Environment="OLLAMA_DEBUG=1"
```

Update

ollamA-webul AI

Machine Translated by Google

Update Ollama by re-running the installation script:

```
curl -fsSL https://ollama.com/install.sh | sh
```

Or by re-downloading Ollama:

```
curl -fsSL https://ollama.com/download/ollama-linux-
amd64.tar.zst \
        | sudo tar x -C /usr
```

Installing specific versions

Use the OLLAMA_VERSION environment variable in your installation script to install a specific version of Ollama, including pre-release versions. Numbers versions can be found on the releases page . For example:

```
curl -fsSL https://ollama.com/install.sh | OLLAMA_VERSION=0.30.0
sh
```

View logs

To view logs of Ollama running as a service, run the following command:

```
journalctl -e -u ollama
```

Removing Ollama

Remove Ollama service:

```
sudo systemctl stop ollama
```

ollamA-webui AI

Machine Translated by Google

```
sudo systemctl disable ollama
sudo rm /etc/systemd/system/ollama.service
```

Remove the Ollama libraries from the lib directory (either /usr/local/lib, /usr/lib, or /lib):

```
sudo rm -r $(which ollama | tr 'bin' 'lib')
```

Remove the ollama executable from the bin directory (either /usr/local/bin, /usr/bin, or /bin):

```
sudo rm $(which ollama)
```

Remove the downloaded models and the Ollama service user and group:

```
sudo userdel ollama
sudo groupdel ollama
sudo rm -r /usr/share/ollama
```

OllamA-webui AI

When choosing a video card, you need to remember that it should not be weaker than CC 5.0 (Compute Capability 5.0+)

The video card must have a microarchitecture no lower than **Maxwell** (for example, the NVIDIA series gtx1060 and above for 4-6 GB models and have the required amount of VRAM for ALL tasks, including providing other programs and processes with VRAM allocation)

When selecting a driver, support for CUDA 12 or 13 must be declared.

(NVIDIA drivers version **535 and higher)**

Machine Translated by Google

Download the version of the model you're interested in at https://ollama.com/search find
the required model by selecting it from the popular ones or by entering a search, for example, "qwen3.5"

Click on the link qwen3.5
(https://ollama.com/library/qwen3.5)

Scroll down the page and select View all ÿ

And the page will appear like this:

...	...	...	...
qwen3.5:4b 2a654d98e6fb 2 months ago	3.4GB	256K	Text, Image
qwen3.5:9b latest 6488c96fa5fa 2 months ago	6.6GB	256K	Text, Image
...	...	...	...
qwen3.5:2b-bf16 64148169aba8 1 month ago	4.6GB	256K	Text, Image
...	...	...	...
qwen3.5:122b-a10b-q4_K_M 8b9d11d807c5 · 2 months ago	81GB	256K	Text, Image

Typical gaming cards have 6-8 Gb of VRAM.

When loading into VRAM, the model takes not only its size, but also the memory under context, which, depending on the settings, can take up an additional 0.5-2 GB VRAM depending on its settings (how to adjust will be described later).

If all this goes beyond the available VRAM, then RAM is already used and The PC CPU starts to slow down when generating a response and takes up program resources and PC processes. Sometimes it's even split between ½ GPU and ½ CPU.

Therefore, we will choose a model that fits the listed parameters.

ollamA-webul AI

Machine Translated by Google

Click on the link qwen3.5:2b-bf16 (4.6GB; 256K; Text, Image) and we will see:

qwen3.5:2b-bf16

11.9M Downloads Updated 1 month ago

Qwen 3.5 is a family of open-source multimodal models that delivers exceptional utility

and performance.

CLI cURL Python JavaScript

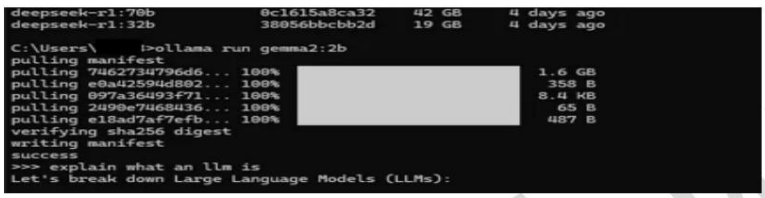
ollama pull qwen3.5:2b-bf16

(pull - save, run - launch)

Let's run it in CLI:

ollama pull qwen3.5:2b-bf16

will look like this (ps: this is from a different 1.6 GB model, not as intelligent)



https://losst.pro/kak-polzovatsya-ollama-dlya-nachinayushhih

- you can follow this link to see all the steps for downloading other things

graphical representation

Let's download all the necessary models using a similar command.

You can see which models are available locally using the ollama command.

list

This will display a list of our models.

Now, let's create an archive of our models for transfer to another server:

(run the command in terminal to pack the entire folder into one compressed archive)

tar -czvf ollama_models_all.tar.gz -C /usr/share/ollama/.ollama models

(the archive name is specified by you and contains all installed models)

ollamA-webul AI

Machine Translated by Google

Downloading Ollama and WebUI Docker

docker pull ollama/ollama

docker pull ghcr.io/open-webui/open-webui:main

(To delete a model, for example, if you don't want to add it to the archive, run:

docker rm model_name) Other commands - refer to the Ollama documentation

Let's save these images to archives:

docker save -o ollama_image.tar ollama/ollama

docker save -o webui_image.tar ghcr.io/open-webui/open-webui:main

NVIDIA Container Toolkit is a special set of tools that allows containers (e.g. Docker or Podman) use a physical graphics card (GPU) of your computer to speed up computing. [1]

NVIDIA Container Toolkit can be downloaded from the following URL:

https://github.com/NVIDIA/nvidia-container-toolkit/releases

Consists of:

NOTE: This release is a unified release of the NVIDIA Container Toolkit that consists of the following packages:

- libnvidia-container 1.19.1
- nvidia-container-toolkit 1.19.1

Download the latest version of the software available at the time of writing the instructions.

https://github.com/NVIDIA/nvidia-container-toolkit/releases/download/v1.19.1/nvidia-container-toolkit_1.19.1_deb_amd64.tar.gz

OllamA-webui AI

Machine Translated by Google

On test server 2:

Let's install the Docker package and add the user to the required group:

```
sudo apt install docker.io
sudo usermod -aG docker $USER
```

About the requirements for the video card and driver (page 7 of this document) and The driver installation (page 4 of this document) was described earlier.

If the driver from the repository is not suitable for some reason, then Install the driver from the manufacturer's website following the instructions Instructions for installing AxxonNext (Intellect-X) FOSS software using the existing driver version suggested by your video adapter.

Starting with the NVIDIA RTX 5xxx series, you must install driver version with the word "open" (nvidia-driver-5xx-open)

Let's install the NVIDIA Container Toolkit

Unzip the downloaded file:

```
tar -xzvf nvidia-container-toolkit_1.19.0_deb_amd64.tar.gz
```

Let's follow the path:

```
cd release-v1.19.0-stable/packages/ubuntu18.04/amd64
```

Let's install the packages:

```
sudo dpkg -i *.deb
```

It is necessary to integrate tools into docker.io

Let's add NVIDIA runtime to Docker's system settings and restart the service:

```
sudo nvidia-ctk runtime configure --runtime=docker
```

OllamA-webui AI

Machine Translated by Google

sudo systemctl restart docker

Let's run the command to import the Ollama image:

sudo docker load -i ollama_image.tar

538812a4b9bd: Loading layer [==>] 80.65MB/80.65MB
f236f023d347: Loading layer [==>] 337.7MB/337.7MB
afceac86a52f: Loading layer [==>] 44.95MB/44.95MB
d2870ea00208: Loading layer [==>] 6.091GB/6.091GB
Loaded image: ollama/ollama:latest

Create a folder for models on the server (if it doesn't exist yet) and unzip yours there.

"scattering" of files from the portable model archive:

sudo mkdir -p /var/lib/docker/volumes/ollama_data/_data/models

sudo tar -xzvf ollama_models_all.tar.gz -C /var/lib/docker/volumes/ollama_data/_data/

(It is recommended to do this with the Docker and Ollama server turned off - we

imported, but not launched yet))

models/
models/blobs/
models/blobs/sha256-832dd9e00a68dd83b3c3fb9f5588dad7dcf337a0db50f7d9483f310cd292e92e
models/blobs/sha256-8893e08fa9f91f7dc39e24d27bdfaece4e9c86bb3269293ff8cea6cba98c872d
models/blobs/sha256-9371364b27a52acac9d87f88bd93c9db1174d8d6ec57f6888925cdc1788871ff
...
models/blobs/sha256-60e05f2100071479f596b964f89f510f057ce397ea22f2833a0cfe029bfc2463
models/blobs/sha256-1e65450c30670713aa47fe23e8b9662bdf4065e81cc8e3cbfaa98924fcc0d320
models/manifests/
models/manifests/registry.ollama.ai/
models/manifests/registry.ollama.ai/library/
models/manifests/registry.ollama.ai/library/llama3.1/
models/manifests/registry.ollama.ai/library/llama3.1/8b
models/manifests/registry.ollama.ai/library/qwen2.5-coder/
models/manifests/registry.ollama.ai/library/qwen2.5-coder/7b
models/manifests/registry.ollama.ai/library/gemma2/
models/manifests/registry.ollama.ai/library/gemma2/9b
models/manifests/registry.ollama.ai/library/qwen2.5/
models/manifests/registry.ollama.ai/library/qwen2.5/7b
models/manifests/registry.ollama.ai/library/deepseek-coder/
models/manifests/registry.ollama.ai/library/deepseek-coder/6.7b-instruct-q5_K_M
models/manifests/registry.ollama.ai/library/qwen3.5/
models/manifests/registry.ollama.ai/library/qwen3.5/4b-q8_0

Let's set the rights: sudo chown -R 755:755 /var/lib/docker/volumes/ollama_data/_data/models

(or) sudo chown -R 997:973 /var/lib/docker/volumes/ollama_data/_data/models

OllamA-Webui AI

Machine Translated by Google

Let's run the command to import the WebUI image:

```
sudo docker load -i open_webui_image.tar
```

Let's create an isolated **docker** virtual network :

```
sudo docker network create ai-cluster-network
```

By default, this is a **bridge** type network and is required for:

Security and Isolation): Containers within the **ai-cluster-network** are completely isolated from other containers in the system that are not part of this network.

Cluster organizations: used to consolidate artificial intelligence infrastructure (for example, Ollama server, Open WebUI web interface and agents) into one logical circuit.

Providing a dedicated name (DNS): Containers on the same user network can communicate with each other with a friend by their names (for example, ***http:// ollama_server:11434),*** and not by IP addresses.

Let's launch the Ollama server:

```
sudo docker run -d \
  --name ollama_server \
  --network ai-cluster-network \
  --gpus all \
  --ipc=host \
  --pids-limit 150 \
  -e OLLAMA_HOST=0.0.0.0 \
  -e OLLAMA_NUM_PARALLEL=1 \
  -e OLLAMA_MAX_LOADED_MODELS=1 \
  -e OLLAMA_KEEP_ALIVE=0 \
  -e OLLAMA_SCHED_SPREAD=false \
  -e NVIDIA_VISIBLE_DEVICES=all \
  -e NVIDIA_DRIVER_CAPABILITIES=compute,utility \
  -v ollama_data:/root/.ollama\
  -p 11434:11434 \
  --restart always \
  ollama/ollama:latest
```

ollamA-webui AI

Machine Translated by Google

Let's see how the Ollama docker started and whether the GPU was passed through:

```
sudo docker logs ollama_server | grep -i -E "gpu|cuda|detect|nvidia"
```

```
time=2026-05-22T09:51:58.557Z level=INFO source=routes.go:1802 msg="server config"
env="map[CUDA_VISIBLE_DEVICES: GGML_VK_VISIBLE_DEVICES: GPU_DEVICE_ORDINAL: HIP_VISIBLE_DEVICES:
HSA_OVE
RRIDE_GFX_VERSION: HTTPS_PROXY: HTTP_PROXY: NO_PROXY: OLLAMA_CONTEXT_LENGTH:0 OLLAMA_DEBUG:INFO
OLLAMA_DEBUG_LOG_REQUESTS:false OLLAMA_EDITOR: OLLAMA_FLASH_ATTENTION:false OLLAMA_GPU_OVER
HEAD:0 OLLAMA_HOST:http://0.0.0.0:11434 OLLAMA_KEEP_ALIVE:0s OLLAMA_KV_CACHE_TYPE:
OLLAMA_LLM_LIBRARY: OLLAMA_LOAD_TIMEOUT:5m0s OLLAMA_MAX_LOADED_MODELS:1 OLLAMA_MAX_QUEUE:512
OLLAMA_MAX_
TRANSFER_STREAMS:4 OLLAMA_MODELS:/root/.ollama/models OLLAMA_MULTIUSER_CACHE:false
OLLAMA_NEW_ENGINE:false OLLAMA_NOHISTORY:false OLLAMA_NOPRUNE:false OLLAMA_NO_CLOUD:false
OLLAMA_NUM_PAR
ALLEL:1 OLLAMA_ORIGINS:[http://localhost https://localhost http://localhost:* https://localhost:* http://127.0.0.1 https://127.0.0.1 http://127.0.0.1:* https://127.0.0.1:*
http://0.0.0.0
https://0.0.0.0 http://0.0.0.0:* https://0.0.0.0:* app://* file://* tauri://* vscode-webview://*
vscode-file://*] OLLAMA_REMOTES:[ollama.com] OLLAMA_SCHED_SPREAD:false OLLAMA_VULKAN:false
ROCR_VISIBLE_DEVICES: http_proxy: https_proxy: no_proxy:]"
time=2026-05-22T09:51:58.557Z level=INFO source=routes.go:1804 msg="Ollama cloud disabled: false"
time=2026-05-22T09:51:58.557Z level=INFO source=images.go:517 msg="total blobs: 0"
time=2026-05-22T09:51:58.557Z level=INFO source=images.go:524 msg="total unused blobs removed: 0"
time=2026-05-22T09:51:58.558Z level=INFO source=routes.go:1864 msg="Listening on [::]:11434
 (version 0.24.0)"
time=2026-05-22T09:51:58.559Z level=INFO source=runner.go:67 msg="discovering available GPUs..."
time=2026-05-22T09:51:58.594Z level=INFO source=server.go:433 msg="starting runner"
cmd="/usr/bin/ollama runner --ollama-engine --port 39275"
time=2026-05-22T09:52:00.095Z level=INFO source=model_recommendations.go:177 msg="model
recommendations cache sleep scheduled" wait=3h51m34.847979847s consecutive_failures=0
time=2026-05-22T09:52:01.542Z level=INFO source=server.go:433 msg="starting runner"
cmd="/usr/bin/ollama runner --ollama-engine --port 44433"
time=2026-05-22T09:52:04.176Z level=INFO source=runner.go:106 msg="experimental Vulkan support
disabled. To enable, set OLLAMA_VULKAN=1"
time=2026-05-22T09:52:04.176Z level=INFO source=server.go:433 msg="starting runner"
cmd="/usr/bin/ollama runner --ollama-engine --port 38377"
time=2026-05-22T09:52:04.300Z level=INFO source=types.go:42 msg="inference compute"
id=GPU-63b55c3a-bf8f-e304-ee7d-19ea877c06d1 filter_id="" library=CUDA compute=6.1
name=CUDA0 description="NVIDIA P106-100" libdirs=ollama,cuda_v12 driver=12.9
pci_id=0000:01:00.0 type=discrete total="6.0 GiB" available="5.9 GiB"
time=2026-05-22T09:52:04.300Z level=INFO source=routes.go:1914 msg="vram-based default context"
total_vram="6.0 GiB" default_num_ctx=4096
```

**Here we see information about our video adapter and additional data*

related to it.

Let's check the availability of models in Docker

ollamA-webui AI

Machine Translated by Google

```
sudo docker exec -it ollama_server ollama list
NAME                                    ID              SIZE    MODIFIED
qwen3.5:4b-q8_0                         8722f47c2791    5.3 GB  30 hours ago
gemma2:9b                               ff02c3702f32    5.4 GB  31 hours ago
llama3.1:8b                             46e0c10c039e    4.9 GB  3 days ago
deepseek-coder:6.7b-instruct-q5_K_M     d6a1221fb5ff    4.8 GB  4 days ago
qwen2.5:7b                              845dbda0ea48    4.7 GB  8 days ago
qwen2.5-coder:7b                        dae161e27b0e    4.7 GB  8 days ago
```

Let's launch the WebUI server:

```
sudo docker run -d \
 --name open_webui \
 --network ai-cluster-network \
 -p 3050:8080 \
 -e WEBUI_NAME="Interface provided by UAR-2020" \
 -e OLLAMA_BASE_URL="http://ollama_server:11434" \
 -e OPENAI_API_BASE_URL="" \
 -e ENABLE_OLLAMA_API=True \
 -e OFFLINE_MODE=true \
 -e HF_HUB_OFFLINE=1 \
 -e WEBUI_AUTH=false \
 -e WEBUI_LOG_LEVEL=WARNING \
 --log-opt max-size=10m \
 --log-opt max-file=3 \
 -v open_webui_data:/app/backend/data\
 --restart always \
 ghcr.io/open-webui/open-webui:main
```

ollamA-webul AI

Machine Translated by Google

```
administrator@docker-test:~$ nvidia-smi
Fri May 22 14:50:41 2026
+-----------------------------------------------------------------------------------------+
| NVIDIA-SMI 575.64.03              Driver Version: 575.64.03      CUDA Version: 12.9     |
|-----------------------------------------+------------------------+----------------------+
| GPU  Name                 Persistence-M | Bus-Id          Disp.A | Volatile Uncorr. ECC |
| Fan  Temp   Perf          Pwr:Usage/Cap |           Memory-Usage | GPU-Util  Compute M. |
|                                         |                        |               MIG M. |
|=========================================+========================+======================|
|   0  NVIDIA P106-100                Off |   00000000:01:00.0 Off |                  N/A |
| 78%   37C    P0             27W /  120W |    5443MiB /   6144MiB |      2%      Default |
|                                         |                        |                  N/A |
+-----------------------------------------+------------------------+----------------------+

+-----------------------------------------------------------------------------------------+
| Processes:                                                                              |
|  GPU   GI   CI              PID   Type   Process name                        GPU Memory |
|        ID   ID                                                               Usage      |
|=========================================================================================|
|    0   N/A  N/A            1771      G   /usr/lib/xorg/Xorg                        4MiB |
|    0   N/A  N/A            5024      C   /usr/bin/ollama                        5436MiB |
+-----------------------------------------------------------------------------------------+
administrator@docker-test:~$
```

In the Ollama configuration file, we have the settings set to load in one
once only one model so that the second one does not load and does not get into the CPU and
slow down the I/O OS (this is a standard situation when there is a lack of GPU, i.e. for normal
We cannot simultaneously load two models with a GPU exceeding 8 GB (on
Some video adapters have 6 GB installed (picture above) and you need to select the model
even more carefully) or you need to install two video cards and a more powerful power supply. Also
instant unloading of the model is provided so that the opponent can do it without delays
connect to your model.

The image above (nvidia-smi output) shows the qwen2.5:7b 4.7 model loading.
GB, which with context processing and generation takes on average 5.5 GB(!!!).
There is no example here of qwen3.5:9b 6.6 GB - when fully deployed it takes up
8.11 GB (!!!) of 8 Gb Nvidia RTX5060 GPU (you need to be more careful, i.e. pay attention
Be careful that the model does not go beyond the GPU in the CPU) and that there is enough GPU for other
resources. The qwen3.5:4b-q8_0 5.3 GB model fits better into the proposed (7.44 GB)

ollamA-webUI AI

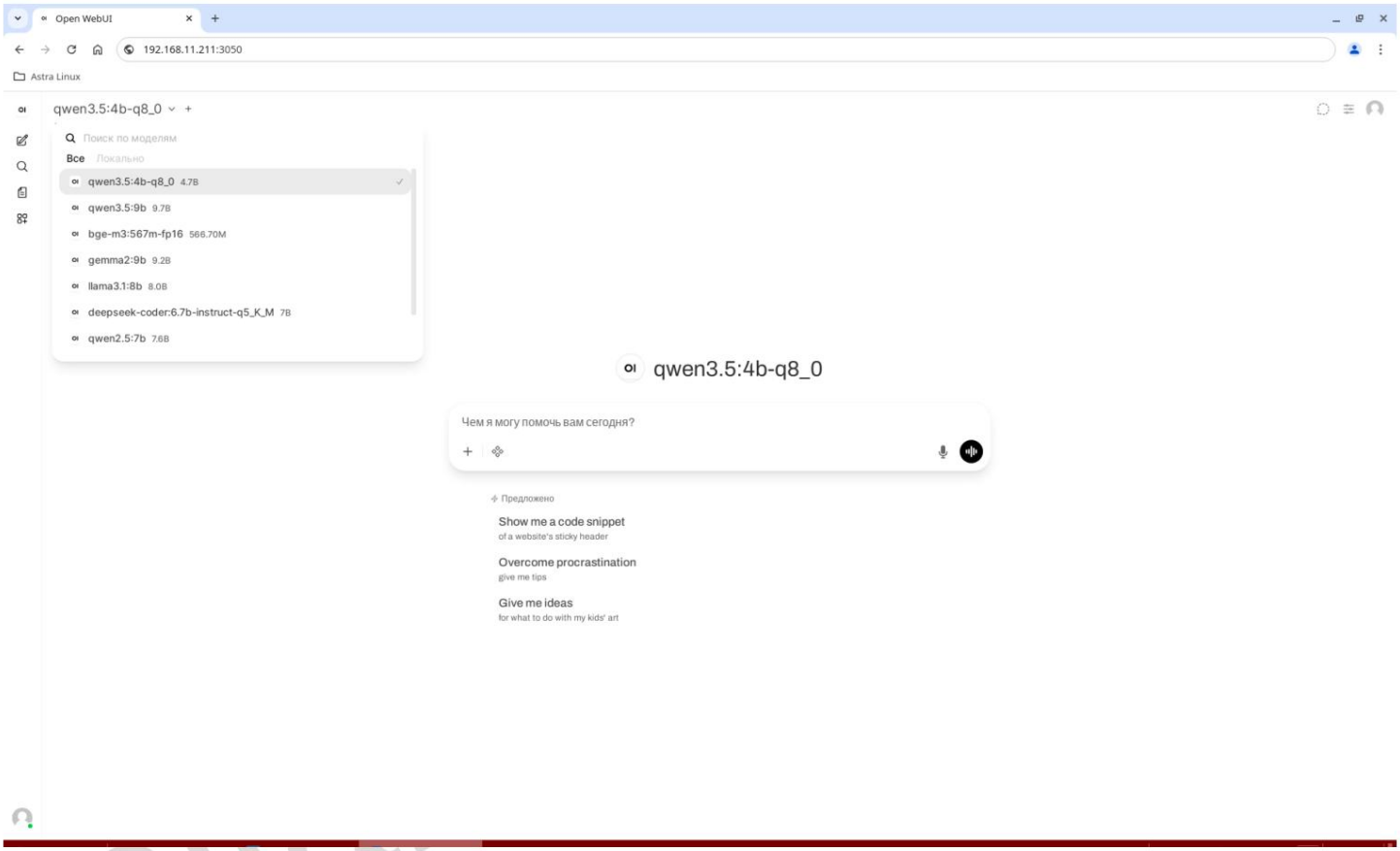
Machine Translated by Google

(at full load), no one forbids using the qwen3.5:9b 6.6 GB model,

You just need to make sure that it doesn't go beyond the GPU (and it does!), because,

For example, a web browser and other software can eat up the GPU we need so much, it is necessary to reach a situation where everyone has enough GPU and there is a little left for possible jumps in GPU usage.

Let's access the mechanism we created through a web browser: IP of our server and port 3050



We select the required model and work (insert text and ask questions).

Some qwen3.5 models can handle images and videos, but for

For this, you need to use another pipeline program (solution found, in development).

You can work from several places at the same time, but the opponent's request will be immediately executed after processing a previously received request. In the settings configuration file we disabled password login to enable it and give

the ability for users to set their own passwords - please refer to

WebUI documentation.

If you get error 500 in your browser, clear your browser (delete web browsing data and cookies).

To be able to access the model (it is suggested to write its name and

Machine Translated by Google

(change as needed) in the CLI we will create a script:

nano ~/.bashrc and add it to the end (correct the IP address in the script to the server's IP address)

Ollamah):

```
ai() {
    echo -e "== Ollama-WebUI AI for CLI AstraLinux [Interface provided by Yuri Arkhipov] =="
    echo -e "Paste the text (Ctrl+Shift+V) and write the questions (or vice versa) and press Enter"
    echo -e "You can immediately type your question and press Enter (pressing Enter several times is acceptable)"
    echo -e "To send to AI, press Enter, then Ctrl+D in any keyboard layout:"
    echo "----------------------------------------------------------------------------------------------"

    # Bind Ctrl+ru/en 'ÿ' to the end-of-input (EOF) signal for bash
    bind '"\Cv": end-of-file' 2>/dev/null
    bind '"\u0432": end-of-file' 2>/dev/null

    # reading the entire text
    local prompt=$(cat)

    # return default key behavior bind '"\Cv": paste' 2>/dev/null


    if [ -z "$(echo "$prompt" | tr -d '[:space:]')" ]; then
        echo "Error: query cannot be empty."
        return 1
    fi
    echo "----------------------------------------------------------------------------------------------"
    echo -ne "\e[1;34mAI:\e[0m "

    # sending to server IP:11434
    jq -n --arg p "$prompt" '{model: "qwen3.5:4b-q8_0", prompt: $p, stream: true}' | \
    curl -s -N -X POST -H "Content-Type: application/json" http://192.168.11.211:11434/api/generate -d @- | \
    jq -j --unbuffered '.response? // empty'

    echo -e "\n"
}
alias fsh=ai
```

ollamA-webul AI

Machine Translated by Google

Once again, PAY ATTENTION TO THE LINE - there should be the IP of the Ollama server

curl -s -N -X POST -H "Content-Type: application/json" http://192.168. 11.211:11434/api/generate…..

It is recommended to enter the name ai in the script instead of the IP and add the following line to /etc/hosts:

ai IP_address_of_Ollama_server (availability check and API response are not provided)

Next, run the command source ~/.bashrc

Let's ask AI, for example, to write a script.

Let's run the script with the ai command

```
administrator@test:~$ ai
== Ollama-WebUI AI for CLI AstraLinux [Interface provided by UAR-2020] ==
    Paste the text (Ctrl+Shift+V) and write the questions (or vice versa) and press Enter
    You can immediately write your question and press Enter (pressing Enter several times is acceptable)
    To send to AI, press Enter, then Ctrl+D in any keyboard layout:
--------------------------------------------------------------------------------
Write a script to check the availability of servers and keep a log when they are unavailable. Their addresses
192.168.11.1, 192.168.11.2, 193.168.11.111
--------------------------------------------------------------------------------
AI: Of course! Below is an example of a Bash script for checking the availability of servers with
specified IP addresses and log entries when unavailable.

 ```bash
#!/bin/bash

# File for storing logs
LOG_FILE="server_log.txt"

# Function for writing to a log file
log_message() {
    echo "$(date +'%Y-%m-%d %H:%M:%S') - $1" >> "$LOG_FILE"
}

# List of server IP addresses
SERVERS=("192.168.11.1" "192.168.11.2" "193.168.11.111")

# Loop to check each server
for SERVER in "${SERVERS[@]}"; do
    # Checking server availability using ping
```

ollama-webui AI
```

Machine Translated by Google

```
        if! ping -c 1 "$SERVER" > /dev/null; then
                log_message "Server $SERVER is unavailable"
        else
                log_message "Server $SERVER is available"
        fi
done

log_message "Verification complete."
```

How to use the script:

1. Save the code to a file, for example `check_servers.sh`.
2. Give it permission to execute:
   ```bash
   chmod +x check_servers.sh
   ```
3. Run the script:
   ```bash
   ./check_servers.sh
   ```

Explanation:

- **log_message**: Function for writing messages with the current date and time to a file `server_log.txt`.
- **SERVERS**: Array of IP addresses of servers to be checked.
- **ping -c 1 "$SERVER" > /dev/null**: The `ping` command sends one data packet to the specified address. The `>` operator redirects output to `/dev/null` to hide the output `ping` commands.
- **if ! ping ... then**: If the server is unavailable (ping failed), then the function is called `log_message` with a message that the server is unavailable.
- **log_message "Check completed."**: At the end of the entry, a message is added to the log file. about the end of the inspection.

This script can be run regularly via cron to continuously monitor availability. servers.
administrator@test:~$

To organize the work of the Ollama server group for distributing requests and

It is proposed to use ngnix for generation, taking into account the GPU load (taking into account

(our settings in idle mode free up the video adapter memory to a minimum).

Some models often take a long time to reason and heat up the video adapter significantly.

ollamA-webul AI

Machine Translated by Google

To disable reasoning in WebUI for a specific model (not all

they argue):

Solution:

1. go to models:

   (face-profile in the upper right corner ÿ Admin Panel ÿ Settings ÿ

Models)

   or type http://your_IP:3050/admin/settings/models

2. Find the desired model and click on the pencil logo to the right of it

   (editing)

3. Find the line “Advanced options” and click “Show” to the right of it.
4. Find the line "think (Ollama)" and set it to "Disabled"
5. At the bottom, find and click the “Save and Update” button.

To disable reasoning in the called ai script:

For example, qwen3.5:4b-q8_0 reasons by default and a lot.

In the CLI, the answer does not appear for a long time and no reasoning is displayed.

Solution:

For the script on page 18 of this document [calls CLI access to AI]

Add the think: false field to the JSON packet generation string.

```
jq -n --arg p "$prompt" '{model: "qwen3.5:4b-q8_0", think: false, prompt: $p, stream: true}'
```

Run source ~/.bashrc

Ollama will now forcefully skip the thought generation step for the specified model.

and will immediately start outputting a text response to the terminal.

The default ai script for the cli (on page 18 of this document) specifies qwen3.5:4b-q8_0. *Choose your model based on the recommendations in this manual (so that there is enough resources to all OS consumers).*

OllamA-webui AI

Machine Translated by Google

Sometimes, you need to start from scratch.

Below are the commands that will provide us with this:

Deleting Ollama and its data

Solution:

Remove Ollama docker:

sudo docker stop ollama_server && sudo docker rm ollama_server

(or remove without stopping the docker with the command "sudo docker rm -f ollama_server")

Remove only models (done after removing the Ollama docker):

sudo rm -rf /var/lib/docker/volumes/ollama_data/_data/models/*

And then, if necessary, erase absolutely all Ollama data

(including logs, metadata and settings):

sudo docker volume rm ollama_data

Removing Open-WebUI and its data:

Solution:

Remove Open-WebUI docker:

sudo docker stop open_webui && sudo docker rm open_webui

(or delete without stopping the docker with the command "sudo docker rm -f open_webui")

And then, if necessary, erase absolutely all Open- data.

WebUI

(including logs, metadata and settings):

sudo docker volume rm open_webui_data

ollamA-webui AI

Machine Translated by Google

On Astra Linux 1.7, memory consumption is related to the specifics of the disk cache.

operating system (Page Cache) and how Docker isolates processes.

When Ollama starts, it reads the model file from disk into RAM.
(RAM), and then transfers it to the graphics card's **VRAM .** According to the Linux specification, read data **is permanently stored in RAM as a filesystem cache,** so that subsequent requests don't require disk reads. Since Ollama runs inside a container, Docker attributes this system cache to your container's memory consumption, which is why you see 10 GB (or another value).

Since nvidia-smi confirms that the model is completely located in the video card, it is possible
Safely remove excess RAM from a container directly when running via Docker
run.

Solution: Limit RAM via the docker run flag

Since we're launching the container with a single command, we'll add the --memory (or -m) flag to it and limit memory consumption to, say, **3-4 GB.** This is enough to run the Ollama API server, but
will prevent the system from inflating the cache.
Just in case, it is suggested to limit RAM consumption at the model size level, but
the necessity of this remains to be studied.

Modify your startup command as follows:

bash

```
sudo docker run -d \
  --name ollama_server \
  --network ai-cluster-network \
  --gpus all \
 -m 4g \
 --memory-swap 4g \
    -...
```

Description of flags:

- -m 4g — strictly prohibits the container and its processes from using more than 4 GB of regular RAM.
memory.
- --memory-swap 4g — disables the ability to dump container data to the swap partition on the disk.
(the value must be equal to the -m value). This is critical on Astra Linux to prevent the container from starting
slow down while trying to write the cache to disk.

LET'S CHECK:

1. Restart the container with new flags (by deleting the docker - see page 22)
2. Let's check the actual memory consumption of the container using the command:

```
docker stats ollama_server
```

Now the MEM USAGE indicator will be fixed within the limit and will not increase. For speed generation (tokens per second) this will not affect in any way, since the calculations themselves are ongoing Exclusively in VRAM. This can be applied to other Dockers.

ollamA-webul AI